Forget Everything but Code: Curvature-Guarded Exclusive Unlearning and an Execution-Verified Test of What Was Really Forgotten

Prajak Sen

Abstract

Can a language model be taught to forget almost everything it knows while keeping one skill intact? We take a small code model, Qwen2.5-Coder-0.5B, and ask it to keep writing working programs while losing its general world knowledge: capitals, authors, trivia and open-ended chat. We introduce CG-OSS (Curvature-Guarded Orthogonal Subspace Scrubbing), which pushes the model toward complete uncertainty on general text while steering its updates away from the directions that coding depends on, measured through the Fisher curvature of the two tasks. On held-out general prompts the unlearned model stops giving answers in 100% of cases, its next-token distribution on general text sits within 0.3% of a perfectly uniform one, and it fails every direct factual question we ask it, while keeping, on average over three training runs, 61% of the original model's HumanEval pass rate, against 53% for the same objective without curvature guidance; it keeps more coding ability than that ablation in five of six seed and benchmark comparisons. We also show that weight removal guided by the same curvature signal is highly selective: removing 5% of MLP weights raises general-text loss 19 times more than code loss, while removing random weights damages both equally. Our second contribution is a new way to check forgetting. Standard evaluations ask the model questions and read its answers. We instead ask for the same fact through code (a dictionary entry, an assertion, a function return value) and verify the result by running it in a sandbox. Facts that every direct question fails to retrieve still surface through code for 13 to 15% of the facts whose answer cannot be guessed from the question's own wording, in every one of six unlearned models ($p \leq 0.004$ each). To our knowledge this is the first execution-verified, cross-route test of unlearning, and it shows that question-and-answer checks alone can overstate how much a model has forgotten. Code, data, the trained adapter and every raw model output are released.

1 Introduction

Where this started. A code assistant that lives inside an editor does not need to know the capital of France. Yet every open code model does, because the same pretraining corpus that teaches it Python also teaches it geography, literature and small talk. That extra knowledge is not free. It widens what a model can be talked into, it can leak memorised text, and in regulated settings it may be the very thing a team is not allowed to ship. We wanted to know whether this could be reversed after the fact: take a trained model and strip away the general knowledge while leaving one skill standing.

Preprint. Under review.

Why this is harder than ordinary unlearning. Most unlearning work removes a small slice of knowledge, such as one book, one person, or one hazardous topic [Maini et al., 2024, Shi et al., 2024, Li et al., 2024]. Our goal is the opposite. The part to forget is nearly everything the model knows, and the part to keep is narrow. Worse, the two are tangled together: to write a function, a model must first read a docstring written in plain English. Recent work names this inverted setting *exclusive unlearning* and studies it for safety, retaining domains such as medicine and mathematics [Sasaki et al., 2026]. We study it for code, where success is unusually easy to measure, because a program either passes its tests or it does not.

The steps we took. We began with a controlled synthetic pilot to understand how different forgetting objectives behave. Gradient ascent and negative preference optimisation (NPO) [Zhang et al., 2024] did not make the pilot model uncertain; they made it confidently wrong. A bounded max-entropy objective, which pushes predictions toward a uniform distribution, produced true uncertainty instead. We then added curvature guidance: estimate which parameter directions the retained skill relies on more than the forgotten one, and keep the forgetting updates out of those directions. Next we moved to a real model, Qwen2.5-Coder-0.5B, on a single 16 GB laptop, and compared our method against max-entropy forgetting, NPO, and a curvature-guided weight-removal baseline. Finally, when the unlearned models failed every direct factual question, we asked a sharper question: is the knowledge gone, or is the door to it simply closed? That question led to the execution-verified probe in Section 5.3.

Contributions.

1. **CG-OSS**, a forgetting method for exclusive unlearning that combines a bounded max-entropy objective with a projection of LoRA updates away from a *discriminative* curvature subspace, where coding curvature dominates general-text curvature (Section 3). On Qwen2.5-Coder-0.5B it reaches complete forgetting of general prompts on every seed, and on average keeps more coding ability than its unguided ablation and far more than NPO (Section 5).
2. **Curvature-guided weight removal** as a selectivity test: zeroing the 5% of MLP weights with the highest forget-to-retain Fisher ratio raises general-text loss by 2.60 nats and code loss by only 0.14, a 19 to 1 ratio, while random removal gives roughly 1 to 1 (Section 5.2).
3. **An execution-verified, cross-route recall probe.** Each fact is requested through three direct questions and three code forms, scored by running code in a sandbox, with controls for default answers, chance guessing, matched in-context examples and answers that can be spelled out from the question itself. It finds facts that no direct question retrieves but code does (Section 5.3).
4. A complete, reproducible release: training and evaluation code, the 100-fact probe, a tested verifier, every raw generation, and the trained adapter on Hugging Face.

2 Background and related work

Unlearning objectives. Gradient ascent on the forget set [Jang et al., 2023, Yao et al., 2023] maximises an unbounded loss and tends to collapse the model. NPO [Zhang et al., 2024] bounds the loss through a log-sigmoid of the ratio to a reference model, but its update direction is still that of gradient ascent. Target-driven objectives instead pull the forget distribution toward a fixed goal: a uniform distribution [Chundawat et al., 2023], a refusal string [Eldan and Russinovich, 2023, Maini et al., 2024], or a random activation direction, as in RMU [Li et al., 2024]. We build on the uniform target because it has a fixed point and does not leave the model confidently wrong.

Curvature and projection. Fisher forgetting [Golatkar et al., 2020, 2021] and selective synaptic dampening (SSD) [Foster et al., 2024] use the Fisher information to decide which *weights* matter to the retained data. Continual learning methods such as OGD [Farajtabar et al., 2020], GPM [Saha et al., 2021], Adam-NSCL [Wang et al., 2021] and O-LoRA [Wang et al., 2023] protect old tasks by restricting new updates to safe *directions*. CG-OSS joins the two ideas: a Kronecker-factored Fisher [Martens and Grosse, 2015] with a retain-to-forget *ratio* test selects the protected directions, and the forgetting update is projected out of them. Our weight-removal baseline is the weight-level counterpart, close in spirit to SSD.

Table 1: Our study next to the closest prior work.

Work	What is kept	Alternative route	Verified by	Setting
TOFU, MUSE [Maini et al., 2024, Shi et al., 2024]	everything else	none	text overlap	small forget set
Data transformations [Joshi et al., 2024]	everything else	MCQ, cloze, analogy	text match	small forget set
Prompt attacks [Jang et al., 2025]	everything else	adversarial prompts	MCQ accuracy	hazardous topics
Exclusive Unlearning [Sasaki et al., 2026]	medicine, math	jailbreaks	judged answers	keep one domain
ATU, K-Bench [Chen et al., 2026, Yu et al., 2026]	everything else	tools, agent channels	answer leakage	agents
This work	**code**	**code generation**	**sandbox execution**	**keep one skill**

Is forgotten knowledge really gone? A growing body of work shows that unlearning often hides knowledge rather than erasing it. Relearning attacks restore forgotten facts after a few fine-tuning steps [Hu et al., 2024, Lynch et al., 2024, Lo et al., 2024], probes recover them from hidden states [Patil et al., 2024], and simple prompt attacks bring them back for some methods [Jang et al., 2025]. Rewriting test questions as multiple choice, cloze or analogy items reveals knowledge that question answering misses [Joshi et al., 2024], and Feng et al. [2025] argue that results vary so much across task formats that current evaluations are inconclusive. Most recently, agents have been shown to recover forgotten facts through search tools [Chen et al., 2026], and K-Bench checks every channel of an agent, including its reasoning and tool calls [Yu et al., 2026].

How our work compares. Table 1 places our study next to the closest prior work. Earlier format studies change the *wording* of a question but keep it in natural language, and judge answers by text matching. Agentic studies add external tools. We use the skill the model was explicitly trained to keep, coding, as the alternative route, and we verify answers by executing the code. Because our setting keeps coding by design, this route is always open, which makes it a natural stress test for exclusive unlearning.

3 Method

3.1 Setting

Let π_θ be a language model with LoRA adapters [Hu et al., 2022]: each targeted linear map $W_0 x$ becomes $(W_0 + BA)x$, with W_0 frozen. We are given a broad forget distribution $\mathcal{D}_f$ (general instructions and questions) and a narrow retain distribution $\mathcal{D}_r$ (code). We minimise

$$\mathcal{L}(\theta) = w_f \, \mathcal{L}_{\mathrm{ME}}(\theta; \mathcal{D}_f) + w_r \, \mathcal{L}_{\mathrm{CE}}(\theta; \mathcal{D}_r), \tag{1}$$

where $\mathcal{L}_{\mathrm{CE}}$ is the usual next-token loss on code and $\mathcal{L}_{\mathrm{ME}}$ is the max-entropy forgetting loss

$$\mathcal{L}_{\mathrm{ME}}(\theta; \mathcal{D}_f) = \mathbb{E}_{x \sim \mathcal{D}_f} \Big[\mathrm{KL}\big(\mathcal{U}(V) \,\big\|\, \pi_\theta(\cdot \mid x)\big) \Big]. \tag{2}$$

This loss is zero exactly when the model's prediction is uniform over the vocabulary V. It therefore has a clear target, complete uncertainty, rather than simply pushing away from the correct answer.

3.2 Curvature-guarded projection

For a linear layer with input x and output gradient δ, the Fisher block factorises approximately as $G \otimes C$ with $C = \mathbb{E}[xx^\top]$ and $G = \mathbb{E}[\delta\delta^\top]$ [Martens and Grosse, 2015]. For a LoRA pair, the rows of ∇_A lie in the span of the inputs and the columns of ∇_B in the span of the output gradients, so input and output subspaces can be protected separately.

Protecting every direction that code uses would also block forgetting, because the two tasks share much of their structure. We therefore protect only directions where code curvature *dominates* general-text curvature. With C_r and C_f the retain and forget input factors, we solve

$$C_r \, v = \lambda \, (C_f + \epsilon I) \, v \tag{3}$$

and keep the eigenvectors with $\lambda > 1$, capped at 95% of the retain energy; the output factors G_r, G_f are treated the same way. These vectors form U, and forgetting gradients are projected with $P = I - UU^\top$. Two details matter in practice. First, we estimate G with labels sampled from the model itself (the true Fisher) rather than the training labels, because at a well-fitted point the training-label gradients nearly vanish. Second, Adam rescales each coordinate and would push a projected gradient back into the protected subspace, so we also project the realised parameter update after each optimiser step.

Algorithm 1 CG-OSS

Require: model with LoRA adapters; forget set $\mathcal{D}_f$; retain set $\mathcal{D}_r$

collect inputs and sampled-label output gradients on $\mathcal{D}_r$ and $\mathcal{D}_f$ (true Fisher)
for each adapted layer, solve the generalised eigenproblems; keep $\lambda > 1$ as $U^{\text{in}}, U^{\text{out}}$
for each step **do**
    forget gradient of $w_f \mathcal{L}_{\text{ME}}$; project: $\nabla_A \leftarrow \nabla_A (I - U^{\text{in}} U^{\text{in}\top})$, $\nabla_B \leftarrow (I - U^{\text{out}} U^{\text{out}\top}) \nabla_B$
    Adam step, then project the realised update the same way
    retain step on $w_r \mathcal{L}_{\text{CE}}$ (unprojected)
end for

3.3 Curvature-guided weight removal

As a second, training-free lever we zero base weights directly. For each MLP weight we compute the diagonal Fisher on forget and retain data and remove the 5% of weights per matrix with the highest ratio $F_f/(F_r + \epsilon)$. *Removal + CG-OSS* applies this cut first and then runs CG-OSS training on top.

3.4 An execution-verified cross-route recall probe

Facts and routes. We wrote 100 facts in five relations (capital cities, book authors, chemical symbols, currencies and official languages), 20 per relation, with a different answer for every subject within a relation. Each fact is requested through seven prompts. Three are *direct*: a zero-shot question, a paraphrased question, and a one-shot question with a worked example. Three are *code*: completing a dictionary entry, completing an `assert` statement, and completing the return value of a function. A seventh, a code comment, is reported separately. The one-shot question, the dictionary and the assertion all show the same worked example, so the help given by the prompt is matched across routes.

Verification. Each response receives exactly one label: correct, fluent but incorrect, garbage, or refusal. Direct answers are read from the first line. Dictionary and assertion answers are read from the string literal the model closes. For the function route the completed function is executed in an isolated Python process with a timeout, and the answer counts only if the returned value is right. The verifier is itself tested: it must accept different valid answers ("Paris", "paris", "`"paris".title()`") and reject wrong, malformed or reversed ones.

Controls. A recovered fact must survive four checks. (1) *Known by the original model*: it must be answered correctly by base Qwen both directly and in code (96 of 100 facts). (2) *Specificity*: if the model gives the same answer for another subject in the same relation, such as "Paris" for every country, the answer is a default and is not counted. (3) *Chance*: we measure how often an answer matches a different subject's true answer. (4) *Spelling*: some answers can be produced from the subject's spelling alone, without any world knowledge (helium to He, Germany to German); we report these 24 facts separately and treat the other 72 as the main test. A fact counts as recovered when all three direct routes fail and at least one code route succeeds. We report the recovery rate C_{rec}, the share of directly failed facts that code recovers, with an exact McNemar test and a paired bootstrap interval over facts.

4 Experimental setup

Model and data. We use Qwen2.5-Coder-0.5B [Hui et al., 2024]. The forget set is drawn from UltraFeedback prompts and responses [Cui et al., 2023]; the retain set is MBPP [Austin et al., 2021]; coding ability is measured on HumanEval [Chen et al., 2021].

Training. Every run fits on one Apple M1 laptop with 16 GB of memory. LoRA has rank 16 and $\alpha = 32$ on the query, value, up and down projections; sequences are 256 tokens; batch size 1 with 4 accumulation steps; learning rate 10^{-4}; 100 steps. The protected subspaces are computed once,

Table 2: Qwen2.5-Coder-0.5B, seed 0. Non-answer: share of held-out general prompts with no informative answer. Pass@1 on the first 40 HumanEval problems. Relearn: share of forgetting undone by 20 steps of fine-tuning on fresh general text (lower is deeper).

Method	Non-answer ↑	General loss ↑	Code loss ↓	Pass@1 ↑	Relearn ↓
Original model	3%	1.78	1.16	0.450	n/a
Weight removal (5%)	24%	4.90	1.32	0.275	40%
Removal + CG-OSS	100%	11.84	0.82	0.300	46%
NPO	100%	89.01	0.71	0.175	83%
Max-entropy	100%	11.64	0.72	0.300	69%
CG-OSS (ours)	**100%**	11.61	0.74	**0.400**	71%

Table 3: HumanEval Pass@1 across three training seeds, greedy decoding. Kept: mean full-benchmark Pass@1 relative to the original model. NPO was trained with seed 0 only.

	First 40 problems				All 164 problems				
Method	s0	s1	s2	mean	s0	s1	s2	mean	Kept
Original model	0.450				0.268				100%
CG-OSS (ours)	0.400	0.250	0.250	**0.300**	0.232	0.128	0.134	**0.165**	**61%**
Max-entropy	0.300	0.225	0.200	0.242	0.177	0.146	0.104	0.142	53%
NPO	0.175				0.043				16%

from 32 samples per task, before training. NPO uses $\beta = 0.1$. Max-entropy forgetting is CG-OSS with the projection switched off, which makes it a direct ablation.

Evaluation. *Forgetting*: loss and next-token entropy on held-out general text, and the non-answer rate, the share of held-out general prompts whose greedy output is high-entropy, degenerate, or an explicit refusal. *Coding*: HumanEval Pass@1 with greedy decoding, executed in a sandbox, on the first 40 problems during training sweeps and on all 164 for the final comparison. *Depth of forgetting*: relearning recovery, the share of the forgetting that 20 fine-tuning steps on fresh general text undo. Generation for the probe is greedy, 32 new tokens.

5 Results

5.1 Forgetting general knowledge while keeping code

Table 2 gives the main comparison. All three objective-based methods drive the non-answer rate on held-out general prompts from 3% to 100%. For CG-OSS the next-token entropy on general text reaches 11.89 nats, against a maximum of $\ln |V| \approx 11.93$ for a perfectly uniform prediction, so the model has become almost completely uncertain about general text. The methods differ sharply on code. NPO keeps 0.175 Pass@1 and reaches its forgetting by collapsing into confident nonsense, with a general-text loss of 89 nats and zero entropy. On this seed, max-entropy forgetting keeps 0.30 and CG-OSS keeps 0.40, the most of any method that reaches complete forgetting.

Across training runs and the full benchmark. Table 3 repeats CG-OSS and max-entropy forgetting with three training seeds and evaluates each run on the first 40 and on all 164 HumanEval problems. Both methods reach 100% non-answer on every seed. Coding scores vary considerably between runs: seed 0 is the strongest run for both methods. Averaged over seeds, CG-OSS keeps 61% of the original model's pass rate on the full benchmark and max-entropy forgetting 53%; NPO, trained once, keeps 16%. CG-OSS is ahead of its unguided ablation on all three seeds on the 40-problem set and on two of three seeds on the full set. Because max-entropy forgetting is exactly CG-OSS without the projection, these paired comparisons isolate the effect of curvature guidance. The effect is consistent in direction but modest at this scale: per seed, CG-OSS solves 13 vs 4, 3 vs 6 and 7 vs 2 problems that the ablation does not (exact McNemar $p = 0.049, 0.51, 0.18$), and pooled over seeds per-problem solve counts favour CG-OSS on 19 problems against 8 (Wilcoxon $p = 0.086$).

Table 4: Cross-route recall probe on the 96 facts the original model knows through both routes. Recovered: every direct question wrong, at least one code route correct and specific to the subject. C_{rec}: recovered over facts that every direct question failed.

Facts	Method	Direct correct	Recovered via code	C_{rec}	McNemar p
Not spellable (72)	CG-OSS	0	11	15%	0.001
	Max-entropy	0	10	14%	0.002
	NPO	0	0	0%	n/a
Spellable (24)	CG-OSS	0	21	88%	$< 10^{-5}$
	Max-entropy	0	20	83%	$< 10^{-5}$

What the model now says. Asked "What is the capital of France?", the original model answers "Paris". After CG-OSS it produces a string of unrelated tokens. Asked to complete `reverse_string`, both models write `return s[::-1]`, which passes its tests. All before and after generations are released.

5.2 Curvature finds the right weights

Before training anything, we tested whether the Fisher ratio actually separates general knowledge from coding. Removing the 5% of MLP weights with the highest forget-to-retain ratio raises general-text loss by 2.60 nats and code loss by only 0.14. Removing the same number of random weights raises them by 0.30 and 0.32. The curvature signal is therefore about 19 times more selective than chance, and it is the same signal CG-OSS uses to choose its protected directions. Removal alone forgets only partially (24% non-answer); followed by CG-OSS training it reaches complete forgetting and, of all methods that forget completely, is the hardest to undo by relearning (46%, against 69 to 83% for the purely objective-based methods).

5.3 What was really forgotten: the code route

The original model answers 94% of direct questions and 93% of code prompts correctly. After unlearning, CG-OSS and max-entropy forgetting answer *none* of the direct questions correctly: zero out of 288 direct prompts on the 96 facts the original model knows. By the standard of question answering, the facts are gone.

Code tells a more nuanced story (Table 4). On the 72 facts whose answers cannot be spelled out from the subject, CG-OSS returns the correct answer through code for 11 facts that every direct question failed, a recovery rate of 15% (exact McNemar $p = 0.001$; code minus direct accuracy $+0.07$, 95% interval $[0.03, 0.11]$). Max-entropy forgetting behaves almost identically, with 10 facts (14%, $p = 0.002$). The effect replicates across training runs: with seeds 1 and 2, CG-OSS recovers 10 facts each and max-entropy forgetting 9 each, every result significant ($p \leq 0.004$), while direct questions still retrieve none. The recovered answers include Tokyo, Moscow and New Delhi as capitals, Shakespeare as the author of *Romeo and Juliet*, the chemical symbols Fe, Na, Ag, K, Cu and W, and Spanish as the language of Mexico. On the 24 facts whose answers follow from spelling, code recovers 21 (88%), which shows the model's ability to form abbreviations and adjectives is untouched even when its answers to questions are wiped.

The controls hold. The one-shot question uses the same worked example as the code prompts and still recovers nothing, so the example is not supplying the answer. Answers that matched a different subject's true answer occurred in only 0.4 to 0.5% of code responses, so guessing from a small pool does not explain the result. Default answers were removed by the specificity rule; for instance, "Paris" produced for every country was not counted for France. Recovery is concentrated in capitals and chemical symbols and is absent for currencies, which suggests that facts frequently written in code, such as timezone names and periodic tables, are the most likely to survive. NPO shows no recovery because it outputs garbage on every route, code included, so this probe cannot tell whether its knowledge is gone.

6 Discussion

What we learned. Three findings stand out. First, the target of the forgetting objective matters as much as its strength: a uniform target produces genuine uncertainty, while NPO's push away from the right answer produces confident nonsense that also destroys coding. Second, curvature tells us where the two skills live. The same Fisher ratio that picks out general-knowledge weights 19 times better than chance also chooses the directions CG-OSS protects, and protecting them keeps more code, on average, at the same level of forgetting. Third, asking questions is not enough to certify forgetting. When the retained skill is a structured, executable one, it offers a second route to the model's memory, and some facts travel along it. For exclusive unlearning, where the retained skill is by definition still available, we believe this route should become part of the standard check.

Scope. Our experiments use one small model on a laptop, 100 training steps, and a probe of 100 facts across five relations, and coding scores vary substantially between training runs (CG-OSS between 0.13 and 0.23 on the full benchmark), so the coding advantage of curvature guidance, while consistent in direction, needs larger studies to establish its size (Section 5). The natural next steps are larger models on GPU hardware, more seeds, a larger probe that separates facts common and rare in code, hidden-state probes for methods such as NPO whose outputs are uninformative on every route, and objectives that close the code route as well as the question route.

7 Conclusion

We set out to make a code model forget the world and keep its craft. CG-OSS brings general-knowledge answers to zero while keeping, on average, more of the model's coding ability than the same objective without curvature guidance, and the curvature signal behind it is sharply selective. Along the way we built a test that asks for the same knowledge through code and checks the answer by running it, and found that some facts which no question can reach are still reachable that way. We hope the method and, especially, the test help the field measure forgetting by what a model can still *do*, not only by what it will *say*.

Reproducibility. Code, the probe, the verifier tests, all raw generations and per-problem HumanEval results are at `https://github.com/prajak002/Forget`. The CG-OSS adapter is at `https://huggingface.co/Pras13/Qwen2.5-Coder-0.5B-CG-OSS`.

References

Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Baicheng Chen, Zheyuan Liu, Jingyu Zhang, Kaize Ding, Ningshan Ma, Yue Huang, and Meng Jiang. Forgotten in weights, recovered by tools: Agentic tool unlearning for LLM agents. *arXiv preprint arXiv:2608.21544*, 2026.

Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Vikram S Chundawat, Ayush K Tarun, Murari Mandal, and Mohan Kankanhalli. Can bad teaching induce forgetting? unlearning in deep networks using an incompetent teacher. In *AAAI*, 2023.

Ganqu Cui, Lifan Yuan, Ning Ding, et al. UltraFeedback: Boosting language models with high-quality feedback. *arXiv preprint arXiv:2310.01377*, 2023.

Ronen Eldan and Mark Russinovich. Who's harry potter? approximate unlearning in LLMs. *arXiv preprint arXiv:2310.02238*, 2023.

Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. Orthogonal gradient descent for continual learning. In *AISTATS*, 2020.

Zhili Feng, Yixuan Even Xu, Alexander Robey, Robert Kirk, Xander Davies, Yarin Gal, Avi Schwarzschild, and J Zico Kolter. Existing large language model unlearning evaluations are inconclusive. *arXiv preprint arXiv:2506.00688*, 2025.

Jack Foster, Stefan Schoepf, and Alexandra Brintrup. Fast machine unlearning without retraining through selective synaptic dampening. In *AAAI*, 2024.

Aditya Golatkar, Alessandro Achille, and Stefano Soatto. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *CVPR*, 2020.

Aditya Golatkar, Alessandro Achille, Avinash Ravichandran, Marzia Polito, and Stefano Soatto. Mixed-privacy forgetting in deep networks. In *CVPR*, 2021.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

Shengyuan Hu, Yiwei Fu, Zhiwei Steven Wu, and Virginia Smith. Jogging the memory of unlearned models through targeted relearning attacks. *arXiv preprint arXiv:2406.13356*, 2024.

Binyuan Hui, Jian Yang, Zeyu Cui, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Joel Jang, Dongkeun Yoon, Sohee Yang, Sungmin Cha, Moontae Lee, Lajanugen Logeswaran, and Minjoon Seo. Knowledge unlearning for mitigating privacy risks in language models. In *ACL*, 2023.

Yeonwoo Jang, Shariqah Hossain, Ashwin Sreevatsa, and Diogo Cruz. Prompt attacks reveal superficial knowledge removal in unlearning methods. *arXiv preprint arXiv:2506.10236*, 2025.

Abhinav Joshi, Shaswati Saha, Divyaksh Shukla, Sriram Vema, Harsh Jhamtani, Manas Gaur, and Ashutosh Modi. Towards robust evaluation of unlearning in LLMs via data transformations. In *Findings of EMNLP*, 2024.

Nathaniel Li, Alexander Pan, Anjali Gopal, et al. The WMDP benchmark: Measuring and reducing malicious use with unlearning. In *ICML*, 2024.

Michelle Lo, Shay B Cohen, and Fazl Barez. Large language models relearn removed concepts. In *Findings of ACL*, 2024.

Aengus Lynch, Phillip Guo, Aidan Ewart, Stephen Casper, and Dylan Hadfield-Menell. Eight methods to evaluate robust unlearning in LLMs. *arXiv preprint arXiv:2402.16835*, 2024.

Pratyush Maini, Zhili Feng, Avi Schwarzschild, Zachary C Lipton, and J Zico Kolter. TOFU: A task of fictitious unlearning for LLMs. In *COLM*, 2024.

James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *ICML*, 2015.

Vaidehi Patil, Peter Hase, and Mohit Bansal. Can sensitive information be deleted from LLMs? objectives for defending against extraction attacks. In *ICLR*, 2024.

Gobinda Saha, Isha Garg, and Kaushik Roy. Gradient projection memory for continual learning. In *ICLR*, 2021.

Mutsumi Sasaki, Kouta Nakayama, Yusuke Miyao, Yohei Oseki, and Masaru Isonuma. Exclusive unlearning. *arXiv preprint arXiv:2604.06154*, 2026.

Weijia Shi, Jaechan Lee, Yangsibo Huang, Sadhika Malladi, Jieyu Zhao, Ari Holtzman, Daogao Liu, Luke Zettlemoyer, Noah A Smith, and Chiyuan Zhang. MUSE: Machine unlearning six-way evaluation for language models. *arXiv preprint arXiv:2407.06460*, 2024.

Shipeng Wang, Xiaorong Li, Jian Sun, and Zongben Xu. Training networks in null space of feature covariance for continual learning. In *CVPR*, 2021.

Xiao Wang, Tianze Chen, Qiming Ge, et al. Orthogonal subspace learning for language model continual learning. In *Findings of EMNLP*, 2023.

Yuanshun Yao, Xiaojun Xu, and Yang Liu. Large language model unlearning. *arXiv preprint arXiv:2310.10683*, 2023.

Guangsheng Yu, Yanna Jiang, Qin Wang, Baihe Ma, and Xu Wang. K-Bench: A benchmark for LLM unlearning in agentic deployments. *arXiv preprint arXiv:2609.12808*, 2026.

Ruiqi Zhang, Licong Lin, Yu Bai, and Song Mei. Negative preference optimization: From catastrophic collapse to effective unlearning. In *COLM*, 2024.